

MICROCONTROLLERS

Labo 1

Naslag

Gegenereerd op 29/07/2026 uit tdmts.github.io/Microcontrollers

Inhoud

Programmeerconcepten

[Arrays](#)

Displays

[Het 7-segment display](#)

[Multiplexing](#)

NASLAG

Arrays

Tot nu toe hield elke variabele één waarde bij. Zodra je met acht leds of tien cijferpatronen werkt, wordt dat onwerkbaar. Een array is één variabele die een hele rij waarden bevat, en die je met een lus kan doorlopen.

Ken je de lessen Technisch programmeren (C#), dan is een array te vergelijken met een [List](#). Het grote verschil is dat je bij een array op voorhand vastlegt hoeveel plaatsen er zijn, en dat aantal verandert daarna niet meer.

Een array declareren

Je zet vierkante haakjes achter de naam en geeft de beginwaarden mee tussen accolades:

```
int lijst[] = {10, 4, -1, 0};
```

CPP

Dit is één variabele, [lijst](#), met vier plaatsen erin. Alle plaatsen hebben hetzelfde type, hier [int](#). Je kan dus geen getallen en tekst door elkaar in dezelfde array steken.

Een waarde opvragen of aanpassen

Elke plaats heeft een **index**, en die begint bij 0. In een array van vier plaatsen zijn de geldige indexen dus 0, 1, 2 en 3:

Index	0	1	2	3
Waarde	10	4	-1	0

De array `lijst` plaats per plaats

```
1 int eersteItem = lijst[0]; // 10, de eerste plaats
2 lijst[1] = 5;           // de tweede plaats krijgt de waarde 5
```

CPP

⚠ BUITEN DE ARRAY LEZEN MERKT NIEMAND OP

Schrijf je `lijst[4]` in een array van vier plaatsen, dan lees of schrijf je geheugen dat niet van jou is. De compiler zegt daar niets over en je sketch loopt gewoon door, maar je krijgt een willekeurige waarde terug of je overschrijft een andere variabele. Dat soort fout is achteraf lastig te vinden, dus tel je indexen goed na.

Het aantal plaatsen laten berekenen

Je kan het aantal plaatsen zelf tellen, maar dan moet je dat getal aanpassen telkens je er een waarde bijzet. Laat de compiler het liever uitrekenen:

```
int aantal = sizeof(lijst) / sizeof(lijst[0]);
```

CPP

`sizeof(lijst)` is hoeveel geheugen de hele array inneemt, `sizeof(lijst[0])` hoeveel één plaats inneemt. De deling geeft dus het aantal plaatsen. Voeg je later een waarde toe, dan klopt het getal nog altijd.

Een array doorlopen met een lus

Met een `for-lus` behandel je alle plaatsen zonder de code te herhalen:

```

1  const int ledPins[] = {3, 4, 5, 6};
2  const int aantalLeds = sizeof(ledPins) / sizeof(ledPins[0]);
3
4  void setup()
5  {
6      for (int i = 0; i < aantalLeds; i++)
7      {
8          pinMode(ledPins[i], OUTPUT);
9      }
10 }

```

De tellervariabele van de lus dient meteen als index. Vier pinnen instellen kost zo drie regels in plaats van vier, en bij zestien leds nog altijd drie.

Tweedimensionale arrays

Soms hoort bij elke plaats niet één waarde maar een hele rij. Denk aan een 7-segment display: voor elk cijfer van 0 tot 9 moet je weten welke van de zeven segmenten aan moeten. Dat zijn tien rijen van zeven waarden, en dan gebruik je **twee** paar haakjes:

```

1  // 3 rijen van 2 waarden
2  const int tabel[3][2] = {
3      {1, 2},
4      {3, 4},
5      {5, 6}
6  };

```

Het eerste getal tussen de haakjes is het aantal **rijen**, het tweede het aantal **kolommen**. Je spreekt een waarde aan met twee indexen, in diezelfde volgorde: eerst de rij, dan de kolom.

	kolom 0	kolom 1
rij 0	<code>tabel[0][0] = 1</code>	<code>tabel[0][1] = 2</code>
rij 1	<code>tabel[1][0] = 3</code>	<code>tabel[1][1] = 4</code>
rij 2	<code>tabel[2][0] = 5</code>	<code>tabel[2][1] = 6</code>

Welke waarde staat waar in `tabel`



BIJ EEN TWEEDIMENSIONALE ARRAY MOET JE HET AANTAL RIJEN WEL ZELF SCHRIJVEN

Bij een gewone array mag je de haakjes leeg laten (`int lijst[] = {...}`), want de compiler telt zelf. Bij twee dimensies moet minstens het aantal kolommen erin staan, anders weet hij niet waar een rij ophoudt. Schrijf ze in de praktijk gewoon allebei, dan lees je meteen hoe groot de tabel is.

Rijen en kolommen laten berekenen

Dezelfde `sizeof`-truc werkt ook bij twee dimensies, alleen moet je ze twee keer toepassen:

CPP

```
1 // De hele tabel gedeeld door één rij geeft het aantal rijen
2 const int aantalRijen = sizeof(tabel) / sizeof(tabel[0]);
3
4 // Eén rij gedeeld door één element geeft het aantal kolommen
5 const int aantalKolommen = sizeof(tabel[0]) / sizeof(tabel[0][0]);
```

`tabel[0]` is de eerste rij, en die is zelf een gewone array. Daarom werkt de bekende deling er ook op. Doe je dit, dan blijft je programma kloppen wanneer je later een rij toevoegt of de tabel breder maakt, zonder dat je ergens een getal moet aanpassen.

Een rij uit de tabel gebruiken

De gebruikelijke vorm is een lus over de kolommen, waarbij de rij bepaalt welk geval je behandelt:

CPP

```
1 // Per cijfer: welke van de 7 segmenten moeten aan? 1 = aan.
2 const bool cijferPatronen[10][7] = {
3     {1, 1, 1, 1, 1, 1, 0}, // 0
4     {0, 1, 1, 0, 0, 0, 0} // 1
5     // ... enzovoort tot en met 9
6 };
7
8 void toonCijfer(int cijfer)
9 {
10     for (int i = 0; i < 7; i++)
11     {
12         // rij = het cijfer, kolom = het segment
13         digitalWrite(segPins[i], cijferPatronen[cijfer][i]);
14     }
15 }
```

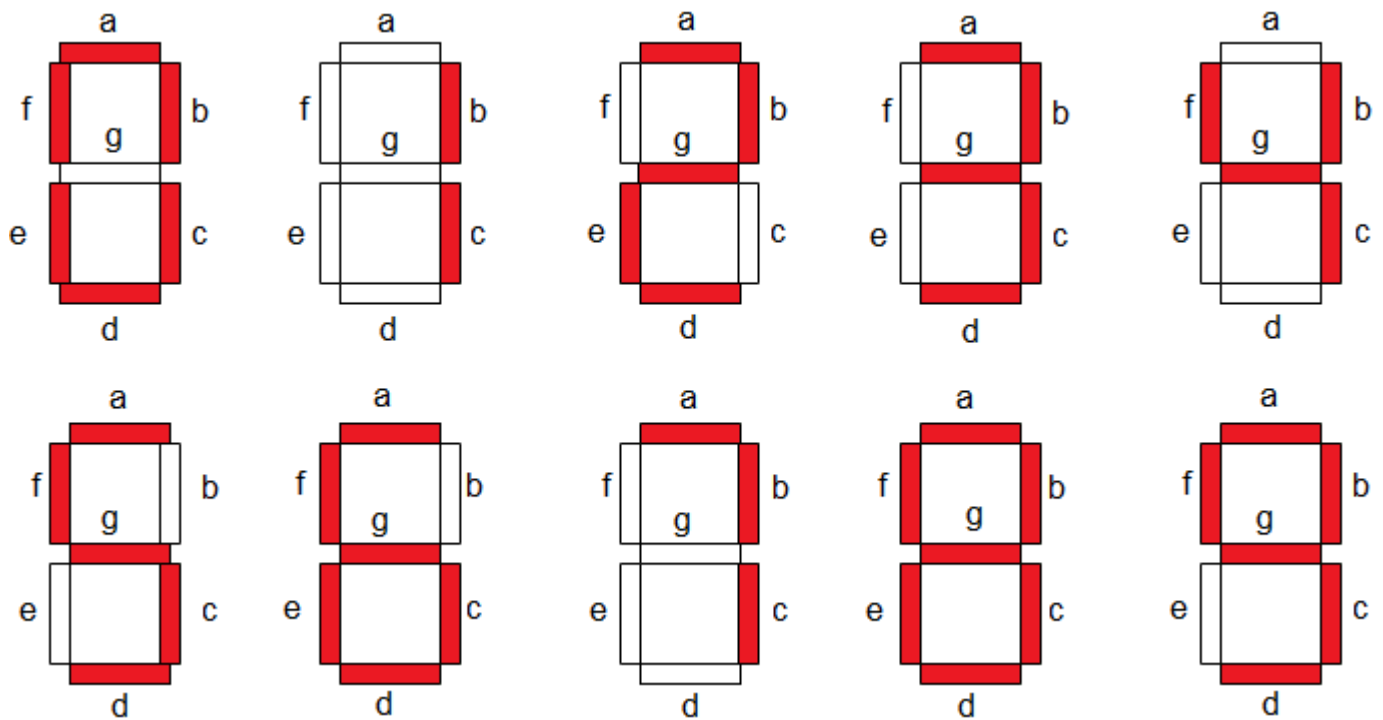
Zonder array zou je hier tien keer zeven `digitalWrite`-regels moeten schrijven, of een selectie met tien takken. Nu staat de kennis in een tabel die je in één oogopslag kan nalezen en aanpassen, en blijft de code die ze gebruikt drie regels lang.

Het 7-segment display

Een 7-segment display is zeven leds in de vorm van een acht, plus meestal een achtste voor de decimale punt. Elk cijfer dat je toont, is een keuze welke van die zeven branden.

De segmenten en hun namen

De zeven segmenten heten a tot en met g, en die namen zijn een afspraak die overal hetzelfde is. Segment a is de bovenste balk, en vanaf daar loop je met de klok mee: b rechtsboven, c rechtsonder, d onderaan, e linksonder, f linksboven, en g de balk in het midden. De decimale punt heet **dp**.



[Klik op de afbeelding om te vergroten](#)

Elk segment heeft zijn eigen aansluiting, en daarnaast is er één **gemeenschappelijke** pin waar alle zeven leds met hun andere kant op uitkomen. Een display sturen kost je dus zeven Arduino-pinnen, en daarom kom je in labo 3 het schuifregister tegen: dat brengt dat aantal terug.

Common anode of common cathode

Er bestaan twee soorten, en welke je hebt bepaalt of je een segment met **HIGH** of met **LOW** laat branden. Het verschil zit in wat er met die gemeenschappelijke pin gebeurt:

	Common cathode	Common anode
De gemeenschappelijke pin gaat naar	GND	+5 V
Een segment brandt bij	HIGH	LOW
De Arduino-pin	sourcet de stroom	sinkt de stroom

De twee soorten 7-segment displays

Het is hetzelfde onderscheid als in [Sourcen en sinken](#), zeven keer naast elkaar.



SCHRIJF HET NIVEAU NOOIT RECHTSTREEKS IN JE CODE

Zet het in een constante bovenaan, dan pas je bij een ander display één regel aan in plaats van je hele programma:

```
1  const bool SEGMENT_AAN = LOW; // common anode; HIGH bij common cathode
2  const bool SEGMENT_UIT = !SEGMENT_AAN;
```

CPP

Weet je niet welk type je hebt, sluit dan één segment aan met zijn weerstand en probeer beide niveaus uit. Wat het segment doet oplichten, is jouw **SEGMENT_AAN**.

⚠ ELK SEGMENT HEEFT ZIJN EIGEN WEERSTAND NODIG

Zeven leds delen hun gemeenschappelijke pin, maar niet hun stroom. Zet je één weerstand in die gemeenschappelijke tak, dan verdeelt de stroom zich over de segmenten die toevallig aan staan en verandert de helderheid met elk cijfer: een 1 (twee segmenten) brandt dan feller dan een 8 (zeven segmenten). Geef daarom elk segment zijn eigen serieweerstand. Hoe je die berekent, staat op [De wet van Ohm](#).

Van cijfer naar patroon

Om een cijfer te tonen moet je weten welke segmenten erbij horen. Voor een 7 zijn dat a, b en c; voor een 0 alles behalve g. Die kennis leg je vast in een [tweedimensionale array](#): één rij per cijfer, één kolom

per segment.

Cijfer	a	b	c	d	e	f	g
0	1	1	1	1	1	1	0
1	0	1	1	0	0	0	0
2	1	1	0	1	1	0	1
3	1	1	1	1	0	0	1

De eerste vier cijfers, met 1 voor "segment aan"

Merk op dat er in die tabel nergens **HIGH** of **LOW** staat. Een 1 betekent "dit segment moet branden", en pas op het moment dat je schrijft, vertaal je dat via **SEGMENT_AAN** naar het niveau dat jouw display nodig heeft. Zo blijft dezelfde tabel bruikbaar voor allebei de soorten.

Meerdere displays

Wil je twee cijfers tonen, dan heb je in principe veertien pinnen nodig. Dat hoeft niet, want je kan de segmentpinnen delen tussen beide displays en razendsnel wisselen welk display je aanstuurt. Hoe dat werkt, staat op [Multiplexing](#).

Multiplexing

Twee cijfers tonen met de pinnen van één display. Je stuurt ze om beurten aan, en je wisselt zo snel dat je oog het verschil niet ziet.

Het probleem

Een **7-segment display** kost je zeven pinnen. Twee displays zouden er dus veertien kosten, en zoveel vrije pinnen heeft een Arduino UNO niet als er ook nog knoppen en leds aan moeten.

Een dubbel display lost dat hardwarematig half op: de zeven segmentaansluitingen van beide cijfers zitten aan elkaar geknoopt. Segment a van het linkse cijfer en segment a van het rechtse hangen aan dezelfde pin. Alleen de **gemeenschappelijke** pin van elk cijfer is apart uitgebracht.

Dat scheelt pinnen, maar het levert meteen een nieuw probleem op: zet je een patroon op de segmentpinnen, dan krijgen *beide* cijfers datzelfde patroon. Je kan dus onmogelijk tegelijk een 4 links en een 7 rechts tonen. Wat je wel kan, is kiezen welk cijfer op dat moment stroom krijgt.

Te snel om te zien

Je toont de twee cijfers dus om beurten:

1. Zet het patroon van het **linkse** cijfer op de segmentpinnen.
2. Activeer alleen het linkse display. Even wachten.
3. Zet het patroon van het **rechtse** cijfer op de segmentpinnen.
4. Activeer alleen het rechtse display. Even wachten.
5. Opnieuw vanaf stap 1.

Doe je dat traag, dan zie je de twee cijfers om beurten flikkeren. Doe je het snel genoeg, dan kan je oog de twee niet meer uit elkaar houden en zie je twee cijfers naast elkaar staan. Dat heet **persistentie van het netvlies**: je oog blijft een lichtpunt nog een fractie van een seconde zien nadat het al gedoofd is. Vanaf ongeveer 50 wisselingen per seconde per cijfer valt er niets meer te merken.

DIT IS DEZELFDE TRUC ALS BIJ FILM

Een film is een reeks stilstaande beelden die snel genoeg na elkaar komen om als beweging over te komen. Bij multiplexing gebeurt hetzelfde, alleen wissel je tussen twee cijfers in plaats van tussen beelden.

In code

Eén doorloop van de wissel ziet er zo uit. Welke stand van de selectiepin welk display activeert, hangt van je schema af, dus zet dat in een constante:

CPP

```
1  const int cijferKiesPin = 10;
2  const bool CIJFER_LINKS = HIGH; // pas aan volgens jouw schema
3
4  void multiplexStap(int linksCijfer, int rechtsCijfer)
5  {
6      digitalWrite(cijferKiesPin, CIJFER_LINKS);
7      toonCijfer(linksCijfer);
8      delay(5);
9
10     digitalWrite(cijferKiesPin, !CIJFER_LINKS);
11     toonCijfer(rechtsCijfer);
12     delay(5);
13 }
```

Met twee keer 5 ms duurt één volledige wissel 10 ms, dus elk cijfer komt honderd keer per seconde aan de beurt, ruim genoeg.

⚠ ER MAG NIETS ANDERS LANG DUREN

Zolang je programma iets anders doet, staat er geen enkel cijfer aan. Zet je ergens een `delay(1000)` om je teller elke seconde op te hogen, dan staat je display die hele seconde donker. Gebruik daarvoor de `millis()-aanpak`: je kijkt telkens of er al genoeg tijd voorbij is, en ondertussen blijft de multiplexing doorlopen.

```
1  void loop()
2  {
3      multiplexStap(teller / 10, teller % 10);
4
5      if (millis() - vorigeTick >= 1000)
6      {
7          vorigeTick = millis();
8          teller = (teller + 1) % 100;
9      }
10 }
```

CPP

💡 HET LIJKT DONKERDER, EN DAT KLOPT

Elk cijfer staat maar de helft van de tijd aan, dus het display brandt minder fel dan wanneer je één cijfer rechtstreeks zou aansturen. Bij twee cijfers valt dat nauwelijks op. Ga je later multiplexen over vier of acht cijfers, dan wordt het wel merkbaar.